

자료구조 실습 보고서

[제 14 주] 사전의 성능 분석 / Call Back 의 구현

제출일 : 2018 년 6 월 21 일

201702081 최재범

1. 프로그램 설명서

a. 주요 알고리즘 / 자료구조

- 이진 탐색 트리

루트를 기준으로, 왼쪽 부트리에는 항상 루트보다 작은 값들이 들어가며 오른쪽 부트리에는 항상 루트보다 큰 값이 들어간다. 이는 모든 루트와 그 부트리에 대하여 성립하므로 재귀적인 구조가 된다. 이진 탐색 트리에서는 값이 중복될 수 없다.

- 사전 (Dictionary)

Key 와 Object 로 이루어지며, 두 값은 서로 쌍을 이루므로 탐색의 과정에서는 Key 를 탐색하여 Object 를 찾는다.

b. 함수 설명서

■ AppController_PerformanceComparement

- public AppController_PerformanceComparement() : 생성자
- public void run() : 프로그램 실행

■ AppController_TreeTraversal

- public AppController_TreeTraversal() : 생성자
- public void run() : 프로그램 실행
- public void visitByCallBack(DictionaryElement<Integer, Integer> anElement, int aLevel) : Key 와 Object 쌍 출력 (Dictionary Model 에 의해 호출되는 Call Back 함수)
- public void visit(DictionaryElement<Integer, Integer> anElement, int aLevel) : Key 와 Object 쌍을 Tree 형태로 역순 출력 (Dictionary Model 에 의해 호출되는 Call Back 함수)

■ Dictionary (추상 클래스)

- public int size() : 현재 크기 반환 (Getter)
- public boolean isEmpty() : 비었는지 확인

■ DictionaryBySortedArray

- public DictionaryBySortedArray() : 최대 크기를 기본값 (100000) 으로 초기화 (생성자)
- public DictionaryBySortedArray(int givenCapacity) : 최대 크기를 givenCapacity 로 초기화 (생성자)
- public int capacity() : 최대 크기 반환 (Getter)
- public boolean isFull() : 다 찼는지 확인
- public boolean keyDoesExist(Key aKey) : Key 에 대응하는 원소 존재 여부 확인
- public Obj objectForKey(Key aKey) : Key 에 대응하는 Object 탐색
- public boolean addKeyAndObject(Key aKey, Obj anObject) : Key 와 Object 쌍 추가

- public Obj removeObjectForKey(Key aKey) : Key 에 대응하는 Object 제거
- public void clear() : 사전 초기화

■ DictionaryBySortedLinkedList

- public DictionaryBySortedLinkedList() : 사전 초기화 (생성자)
- public boolean isFull() : 다 찼는지 확인
- public boolean keyDoesExist(Key aKey) : Key 에 대응하는 원소 존재 여부 확인
- public Obj objectForKey(Key aKey) : Key 에 대응하는 Object 탐색
- public boolean addKeyAndObject(Key aKey, Obj anObject) : Key 와 Object 쌍 추가
- public Obj removeObjectForKey(Key aKey) : Key 에 대응하는 Object 제거
- public void clear() : 사전 초기화

■ DictionaryByBinarySearchTree

- public DictionaryByBinarySearchTree() : 사전 초기화 (생성자)
- public VisitEventSourceForTreeTraversal<Key, Obj> visitEvent() : 저장된 visitEvent 객체를 반환 (Getter)
- public void setVisitEvent(VisitEventSourceForTreeTraversal<Key, Obj> newVisitEvent) : visitEvent 객체를 새로 설정
- public boolean isFull() : 다 찼는지 확인
- public boolean keyDoesExist(Key aKey) : Key 에 대응하는 원소 존재 여부 확인
- public Obj objectForKey(Key aKey) : Key 에 대응하는 Object 탐색
- public boolean addKeyAndObject(Key aKey, Obj anObject) : Key 와 Object 쌍 추가
- public Obj removeObjectForKey(Key aKey) : Key 에 대응하는 Object 제거
- public void clear() : 사전 초기화
- public void scanInSortedOrder() : 중위 탐색
- public void scanReverseOfSortedOrder() : 거꾸로 중위 탐색

■ ListNode

- public ListNode() : 디폴트 생성자
- public ListNode(E givenElement, ListNode<E> givenNext) : 주어진 원소, 다음 노드로 설정 (생성자)
- public E element() : 노드의 원소 반환 (Getter)
- public void setElement() : 노드의 원소 새로 설정 (Setter)
- public ListNode<E> next() : 다음 노드 반환 (Getter)
- public void setNext() : 다음 노드 새로 설정 (Setter)

■ BinaryNode

- public BinaryNode() : 원소, 다음 왼쪽노드, 다음 오른쪽 노드 초기화 (생성자)
- public BinaryNode(E givenElement, BinaryNode<E> givenLeft, BinaryNode<E> givenRight) : 원소, 다음 왼쪽 노드, 다음 오른쪽 노드를 주어진 값으로 초기화 (생성자)

- public E element() : 노드의 원소 반환 (Getter)
- public void setElement() : 노드의 원소 새로 설정 (Setter)
- public BinaryNode<E> left() : 왼쪽 다음 노드 반환 (Getter)
- public void setLeft(BinaryNode<E> newLeft) : 왼쪽 다음 노드 설정 (Setter)
- public BinaryNode<E> right() : 오른쪽 다음 노드 반환 (Getter)
- public void setRight(BinaryNode<E> newRight) : 오른쪽 다음 노드 설정 (Setter)

■ ListOrder (열거형)

- public String toStringInKorean() : 주어진 ListOrder 형의 변수에 맞는 한글 이름 반환

■ DictionaryElement

- public DictionaryElement() : 디폴트 생성자
- public DictionaryElement(Key givenKey, Obj givenObject) : Key, Object 값을 주어진 값으로 설정
- public Key key() : 저장된 Key 값 반환 (Getter)
- public void setKey(Key newKey) : Key 값 새로 설정 (Setter)
- public Obj object() : 저장된 Object 값 반환 (Getter)
- public void setObject(Obj newObject) : Object 값 새로 설정 (Setter)

■ DataGenerator

- public static int[] ascendingList(int aSize) : 주어진 크기 만큼의 오름차순 정수 리스트 생성하여 반환
- public static int[] descendingList(int aSize) : 주어진 크기 만큼의 내림차순 정수 리스트 생성하여 반환
- public static int[] randomList(int aSize) : 주어진 크기 만큼의 무작위 정수 리스트 생성하여 반환

■ ParameterSet

- public ParameterSet(int givenNumberOfExperiments, int givenFirstDataSize, int givenSizeIncrement) : 수행할 실험 수, 첫 데이터 크기, 데이터 크기 증가량을 주어진 값으로 설정 (생성자)
- public int firstDataSize() : 첫 데이터 크기 반환 (Getter)
- public void setFirstDataSize(int newFirstDataSize) : 첫 데이터 크기를 주어진 값으로 설정 (Setter)
- public int numberOfExperiments() : 수행할 실험 수 반환 (Getter)
- public void setNumberOfExperiments(int newNumberOfExperiments) : 수행할 실험 수를 주어진 값으로 설정 (Setter)
- public int sizeIncrement() : 데이터 크기 증가량 반환 (Getter)
- public void setSizeIncrement(int newSizeIncrement) : 데이터 크기 증가량을 주어진 값으로 설정 (Setter)
- public int maxDataSize() : 최대 데이터 크기 반환

■ PerformanceMeasurement

- `public PerformanceMeasurement()` : 수행할 실험 수, 첫 데이터 크기, 데이터 크기 증가량을 현재 클래스 내의 `parameterSet` 필드에 기본값으로 설정 (생성자)
- `public PerformanceMeasurement(int givenNumberOfExperiments, int givenFirstDataSize, int givenSizeIncrement)` : 수행할 실험 수, 첫 데이터 크기, 데이터 크기 증가량을 현재 클래스 내의 `parameterSet` 필드에 주어진 값으로 설정 (생성자)
- `public ParameterSet parameterSet()` : 저장된 `parameterSet` 필드 반환 (Getter)
- `public void setParameterSet(ParameterSet newParameterSet)` : `parameterSet` 필드를 주어진 것으로 새로 설정 (Setter)
- `public int[] ascendingList()` : 저장된 오름차순 리스트 반환 (Getter)
- `public int[] descendingList()` : 저장된 내림차순 리스트 반환 (Getter)
- `public int[] randomList()` : 저장된 무작위 리스트 반환 (Getter)
- `public void generateData()` : 데이터 생성
- `public int[] experimentList(ListOrder anOrder)` : 주어진 List Order 에 맞는 데이터 리스트 반환
- `public UnitExperimentResult[] experimentByDictionaryAndListOrderType(Dictionary<Integer, String> aDictionary, ListOrder anOrder)` : Dictionary 종류와 데이터 종류를 설정하여, 데이터 크기를 변화시키며 측정 실험 실행

■ UnitExperimentResult

- `public UnitExperimentResult()` : 디폴트 생성자
- `public UnitExperimentResult(int givenExperimentSize, long givenTimeForAdd, long givenTimeForSearch, long givenTimeForRemove)` : 수행할 실험 수, 추가에 소요된 시간, 탐색에 소요된 시간, 제거에 소요된 시간을 주어진 값으로 설정
- `public int experimentSize()` : 저장된 수행할 실험 수 반환
- `public void setExperimentSize(int newExperimentSize)` : 수행할 실험 수를 주어진 것으로 새로 설정 (Setter)
- `public long timeForAdd()` : 저장된 추가에 소요된 시간 반환
- `public void setTimeForAdd(long newTimeForAdd)` : 추가에 소요된 시간을 주어진 것으로 새로 설정 (Setter)
- `public long timeForSearch()` : 저장된 탐색에 소요된 시간 반환
- `public void setTimeForSearch(long newTimeForSearch)` : 탐색에 소요된 시간을 주어진 것으로 새로 설정 (Setter)
- `public long timeForRemove()` : 저장된 삭제에 소요된 시간 반환
- `public void setTimeForRemove(long newTimeForRemove)` : 삭제에 소요된 시간을 주어진 것으로 새로 설정 (Setter)

■ AppView

- `public AppView()` : 생성자
- `public void output(String aString)` : 주어진 문자열 출력
- `public void outputLine(String aString)` : 주어진 문자열 줄바꿈 출력

c. 종합 설명서

사전 (Dictionary) 를 배열, 링크드 리스트, 이진탐색트리 의 세 가지 방법으로 나누어 성능을 측정하였으며

또한 이진 탐색 트리의 형태를 출력하는 테스트도 따로 진행하였다.

각각의 AppController 를 만들어, main 에서 따로따로 실행하였다.

2. 프로그램 장단점 분석

장점 : 트리의 구조를 쉽게 확인 가능

단점 : 시간이 오래 걸림

3. 실행 결과 분석

a. 입력과 출력

파일이 너무 많아서 따로 첨부하였음.

b. 결과분석

1. 트리 출력 테스트

출력된 트리를 확인해보면, 루트를 기준으로 왼쪽 부트리에는 더 작은 값들이, 오른쪽에는 더 큰 값들이 있는 이진 탐색 트리의 형태가 올바르게 출력되었음을 확인할 수 있었다.

2. 성능 측정 결과 분석

① 오름차순

- SortedArray

인덱스로 바로 접근 가능하므로 검색 빠름

오름차순이기 때문에 그냥 뒤에다가 삽입하므로 삽입 빠름

삭제할 때는 오름차순 리스트를 오름차순으로 삭제하므로 매 번 인덱스를 배열의 크기만큼 밀어야하기 때문에 시간이 오래걸림

하지만 LinkedList 와 SearchTree 의 삽입에서는 매 번 삭제할 원소까지 탐색을 해야 하기 때문에 후자가 더 오래걸림

- SortedLinkedList

삽입할 때에는 위치를 찾는게 대부분의 시간이고 원소를 추가하는 과정은 얼마 안걸리기 때문에 검색과 시간 차이가 얼마 안남

삭제할 때 오름차순으로 삭제하므로 매 번 root 와 잇기만 하면 되기 때문에 빠름

SortedArray 는 삭제하면서 인덱스를 밀면 되지만 SortedLinkedList 는 삽입할 때마다 매 번 맨 끝까지 이동해야 하므로 훨씬 오래걸림

- BinarySearchTree

삽입할 때에는 위치를 찾는게 대부분의 시간이고 원소를 추가하는 과정은 얼마 안걸리기 때문에 검색과 시간 차이가 얼마 안남

BinarySearchTree 에서 최솟값은 앞에 있으므로 그냥 제거만 하면 되기 때문에 빠름
둘 다 연결체인으로 구성되었기 때문에 전체적으로는 SortedLinkedList 와 비슷함

② 내림차순

- SortedArray

추가할 때마다 매 번 배열의 크기만큼 밀어야 하므로 오래걸림

정렬이 완료된 오름차순 리스트를 내림차순으로 삭제하므로 맨 뒤의 원소만 삭제하면 되기 때문에 삭제 빠름

다른 두 구현은 오름차순 리스트인데 내림차순으로 삭제하기 때문에 매 번 삭제할 때마다 맨 끝까지 이동하므로 훨씬 오래걸림

- SortedLinkedList

삭제하려면 검색을 해야 하는데 삭제 자체는 시간이 얼마 안걸리고 검색하는데 대부분 소요되기 때문에 둘이 비슷함

삽입은 추가할 원소를 맨 앞에다가만 추가하게 되므로 빠름

- BinarySearchTree

검색 후 삽입하는데 삽입 자체는 시간이 얼마 안걸리므로 둘이 비슷함

최댓값은 항상 앞에 있으므로 삭제 빠름

둘 다 연결체인으로 구성되었기 때문에 전체적으로는 SortedLinkedList 와 비슷함

③ 무작위

- SortedArray

삽입과 삭제에 소요되는 시간은 거의 인덱스를 찾는 시간인데 무작위이기 때문에 한 쪽에 치우치지 않고 골고루 걸리게 됨

- SortedLinkedList

매 번 원소에 접근할 때마다 이전의 모든 노드를 거쳐야 하기에 SortedArray 보다 더 오래걸림

- BinarySearchTree

무작위이기 때문에 전체적으로 성능이 $O(\log N)$ 에 근접하는 이진탐색트리의 특성에 따라서 전부 비슷하며 적게 걸림

4. 소스코드

AppController_PerformanceComparement.java

```
public class AppController_PerformanceComparement {  
  
    // 성능 측정 설정에 사용되는 상수  
    private static final int APP_NUMBER_OF_EXPERIMENTS = 5;  
    private static final int APP_FIRST_DATA_SIZE = 10000;  
    private static final int APP_SIZE_INCREMENT = 10000;  
  
    // 인스턴스 변수  
    private AppView _appView;
```

```

private PerformanceMeasurement _measurement;

// 생성자
public AppController_PerformanceComparement() {
    this._appView = new AppView();
}

public void run() {
    this._appView.outputLine("<< \"Dictionary\"의 성능 측정을 시작합니다. >>> \n");
    this._measurement = new PerformanceMeasurement(APP_NUMBER_OF_EXPERIMENTS,
APP_FIRST_DATA_SIZE,
APP_SIZE_INCREMENT);

    this.showExperimentByListOrderType(ListOrder.Ascending);
    this.showExperimentByListOrderType(ListOrder.Descending);
    this.showExperimentByListOrderType(ListOrder.Random);

    this._appView.outputLine("<< \"Dictionary\"의 성능 측정을 종료합니다. >>> \n");
}

// 비공개 함수

// 데이터 종류 설정 후 측정, 출력
private void showExperimentByListOrderType(ListOrder anOrder) {
    this._appView.outputLine("< " + anOrder.toStringInKorean() + " 데이터를 사용한 측정 (단위 : mili second) >");

    this.showExperimentByDictionaryAndListOrderType (
        new DictionaryBySortedArray<Integer, String> (this._measurement.parameterSet().maxDataSize()),
        anOrder );

    this.showExperimentByDictionaryAndListOrderType (
        new DictionaryBySortedLinkedList<Integer, String> (),
        anOrder );

    this.showExperimentByDictionaryAndListOrderType (
        new DictionaryByBinarySearchTree<Integer, String> (),
        anOrder );

    this._appView.outputLine("");
}

// 사전 종류, 데이터 종류 설정 후 측정, 출력
private void showExperimentByDictionaryAndListOrderType( Dictionary<Integer, String> aDictionary,
ListOrder anOrder )
{
    this._appView.outputLine(""" + aDictionary.getClass().getName() + "\"로 구현된 사전의 성능 : ");
    this._appView.output(String.format("%6s", "크기 "));
    this._appView.output(String.format("%11s", "삽입"));
    this._appView.output(String.format("%11s", "검색"));
    this._appView.outputLine(String.format("%11s", "삭제"));
}

```



```

        UnitExperimentResult[] results = this._measurement.experimentByDictionaryAndListOrderType
                                                    (aDictionary, anOrder);

        // 반복 회차별 결과 출력
        for( int iteration = 0;
            iteration < this._measurement.parameterSet().numberOfExperiments();
            iteration++ )
        {
            this.showUnitExperimentResult(results[iteration]);
        }
    }

    // 측정 완료된 결과를 출력
    private void showUnitExperimentResult(UnitExperimentResult aResult) {
        this._appView.output( String.format( "[%5d]", aResult.experimentSize() ) );
        this._appView.output( String.format( "%12d", aResult.timeForAdd() / 1000 ) );
        this._appView.output( String.format( "%12d", aResult.timeForSearch() / 1000 ) );
        this._appView.outputLine( String.format( "%12d", aResult.timeForRemove() / 1000 ) );
    }
}

```

AppController_TreeTraversal.java

```

public class AppController_TreeTraversal
implements VisitEventSourceForTreeTraversal<Integer, Integer>
{
    private static final int DEFAULT_DATA_SIZE = 10;

    private AppView _appView;
    private DictionaryByBinarySearchTree<Integer, Integer> _dictionary;
    private int[] _list;
    private int dataSize;

    public AppController_TreeTraversal() {
        this._appView = new AppView();
    }

    public void run() {
        {
            this.dataSize = DEFAULT_DATA_SIZE;

            this._list = new int [this.dataSize];
            this._list = DataGenerator.randomList(this.dataSize);
            this._dictionary = new DictionaryByBinarySearchTree<Integer, Integer> ();

            this._dictionary.setVisitEvent(this);

            this.addToBinarySearchTreeAndShowShape();
            this.showInOrderOfBinarySearchTree();
            this.removeFromBinarySearchTreeAndShowShape();
        }

        {
            this.dataSize += 10;
        }
    }
}

```

```

        this._list = new int[this.dataSize];
        this._list = DataGenerator.randomList(this.dataSize);
        this._dictionary = new DictionaryByBinarySearchTree<Integer, Integer> ();

        this._dictionary.setVisitEvent(this);

        this.addToBinarySearchTreeAndShowShape();
        this.showInOrderOfBinarySearchTree();
        this.removeFromBinarySearchTreeAndShowShape();
    }

    this._appView.output("\n-----\n\n");
}

// Model 에서 Call Back 할 함수 : Key 와 Object 출력
@Override
public void visitByCallBack(DictionaryElement<Integer, Integer> anElement, int aLevel) {
    this._appView.outputLine(anElement.key() + " (" + anElement.object() + ")");
}

// Model 에서 Call Back 할 함수 : Key 와 Object 를 Tree 형태로 역순 출력
@Override
public void visitInReverseOrder(DictionaryElement<Integer, Integer> anElement, int aLevel) {
    for(int i = 0; i < aLevel; i++) {
        this._appView.output("  ");
    }
    this._appView.outputLine(anElement.key() + " (" + anElement.object() + ")");
}

// 비공개 함수

// 사전에 무작위 원소 추가하며 트리 형태로 역순 출력
private void addToBinarySearchTreeAndShowShape() {

    this._appView.outputLine("<<< 삽입 과정에서의 이진검색트리의 변화 >>>\n");
    this._list = DataGenerator.randomList(this.dataSize);

    for(int i = 0; i < this.dataSize; i++) {
        this._appView.outputLine(this._list[i] + " (" + i + ") 원소를 삽입한 후의 이진검색트리 : ");

        this._dictionary.addKeyAndObject(this._list[i], i);
        this._dictionary.scanReverseOfSortedOrder();
        this._appView.outputLine("");
    }
}

// 사전을 중위 탐색하며 원소 출력
private void showInOrderOfBinarySearchTree() {
    this._appView.outputLine("\n<<< Inorder Traversal >>>");
    this._dictionary.scanInSortedorder();
    this._appView.outputLine("");
}

```

```

// 사전에서 무작위 원소를 제거하며 트리 형태로 역순 출력
private void removeFromBinarySearchTreeAndShowShape() {
    this._appView.outputLine("\n<<< 삭제 과정에서의 이진검색트리의 변화 >>>\n");

    for(int i = 0; i < this.dataSize; i++) {
        this._appView.outputLine("Key 값이 " + this._list[i] + " 인 원소를 삭제한 후의 이진검색트리 : ");

        this._dictionary.removeObjectForKey(this._list[i]);
        this._dictionary.scanReverseOfSortedOrder();
    }
}
}

```

AppView.java

```

import java.util.Scanner;

public class AppView {

    private Scanner _scanner;

    // 생성자
    public AppView() {
        this._scanner = new Scanner(System.in);
    }

    // 문자열 출력
    public void output(String aString) {
        System.out.print(aString);
    }

    // 문자열
    public void outputLine(String aString) {
        System.out.println(aString);
    }
}

```

BinaryNode.java

```

public class BinaryNode<E> {

    // 인스턴스 변수
    private E _element;
    private BinaryNode<E> _left;
    private BinaryNode<E> _right;

    // Getter / Setter
    public E element() { return this._element; }
    public void setElement(E newElement) { this._element = newElement; }

    public BinaryNode<E> left() { return this._left; }
    public void setLeft(BinaryNode<E> newLeft) { this._left = newLeft; }

    public BinaryNode<E> right() { return this._right; }
    public void setRight(BinaryNode<E> newRight) { this._right = newRight; }

    // 생성자
}

```

```

    public BinaryNode() {
        this.setElement(null);
        this.setLeft(null);
        this.setRight(null);
    }
    public BinaryNode(E givenElement, BinaryNode<E> givenLeft, BinaryNode<E> givenRight) {
        this.setElement(givenElement);
        this.setLeft(givenLeft);
        this.setRight(givenRight);
    }
}

```

DataGenerator.java

// 데이터 생성
 // static class를 구현하기 위하여, 모든 멤버는 static으로 선언하고 클래스는 final로

```
import java.util.Random;
```

```
public final class DataGenerator {
```

// 인스턴스 생성 불가

```
private DataGenerator() {}
```

// 오름차순 리스트 생성

```
public static int[] ascendingList(int aSize) {

    if(aSize > 0) {
        int[] list = new int[aSize];
        for(int i = 0; i < aSize; i++) {
            list[i] = i;
        }
        return list;
    }

    return null;
}

```

// 내림차순 리스트 생성

```
public static int[] descendingList(int aSize) {

    if(aSize > 0) {
        int[] list = new int[aSize];
        for(int i = 0; i < aSize; i++) {
            list[i] = aSize - 1 - i;
        }
        return list;
    }

    return null;
}

```

// 무작위 리스트 생성

```
public static int[] randomList(int aSize) {

    if(aSize > 0) {
        int[] list = new int[aSize];
        for(int i = 0; i < aSize; i++) {
            list[i] = i;
        }
    }
}

```

```
Random random = new Random();
```

```

        for(int i = 0; i < aSize; i++) {
            int j = random.nextInt(aSize);
            Integer temp = list[i];
            list[i] = list[j];
            list[j] = temp;
        }
        return list;
    }

    return null;
}
}

```

Dictionary.java

```

public abstract class Dictionary<Key extends Comparable<Key>, Obj> {

    private int _size; // 상속받는 클래스에서 Getter / Setter 로만 접근 가능

    // Getter / Setter
    public int size() { return this._size; }
    protected void setSize(int newSize) { this._size = newSize; } // 삽입/삭제할 때 크기 변경시킬 때 사용

    // 생성자
    public Dictionary() { this.setSize(0); }

    // 비었는 지 확인
    public boolean isEmpty() {
        return ( this.size() == 0 );
    }

    // 추상 메소드
    public abstract boolean isFull();
    public abstract boolean keyDoesExist(Key aKey);
    public abstract Obj objectForKey(Key aKey);
    public abstract boolean addKeyAndObject(Key aKey, Obj anObject);
    public abstract boolean removeObjectForKey(Key aKey);
    public abstract void clear();
}

```

DictionaryByBinarySearchTree.java

```

public class DictionaryByBinarySearchTree <Key extends Comparable<Key>, Obj>
extends Dictionary <Key, Obj>
{

    // 인스턴스 변수
    private BinaryNode<DictionaryElement<Key, Obj>> _root;
    private VisitEventSourceForTreeTraversal<Key, Obj> _visitEvent; // 전달받은 Visit Event 객체를 저장할 곳

    // Getter / Setter
    private BinaryNode<DictionaryElement<Key, Obj>> root() { return this._root; }
    private void setRoot(BinaryNode<DictionaryElement<Key, Obj>> newRoot) { this._root = newRoot; }

    public VisitEventSourceForTreeTraversal<Key, Obj> visitEvent() { return this._visitEvent; }
    public void setVisitEvent(VisitEventSourceForTreeTraversal<Key, Obj> newVisitEvent) { this._visitEvent = newVisitEvent; }
}

```

```

// 생성자
public DictionaryByBinarySearchTree() {
    this.clear();
}

// 다 찼는지 확인
@Override
public boolean isFull() {
    return false;
}

// Key 의 존재 여부 확인
@Override
public boolean keyDoesExist(Key aKey) {
    return ( this.elementForKey(aKey) != null );
}

// Key 에 대응하는 Object 탐색
@Override
public Obj objectForKey(Key aKey) {
    DictionaryElement<Key, Obj> element = this.elementForKey(aKey);
    if(element != null)
        return element.object();
    else
        return null;
}

// Key 와 Object 쌍 추가
@Override
public boolean addKeyAndObject(Key aKey, Obj anObject) {

    DictionaryElement<Key, Obj> elementForAdd =
        new DictionaryElement<Key, Obj> (aKey, anObject);
    BinaryNode<DictionaryElement<Key, Obj>> nodeForAdd =
        new BinaryNode<DictionaryElement<Key, Obj>> (elementForAdd, null, null);

    // 비어있을 때 추가
    if(this.root() == null) {
        this.setRoot(nodeForAdd);
        this.setSize(1);
        return true;
    }

    // 중간에 추가
    BinaryNode<DictionaryElement<Key, Obj>> current = this.root();

    while(aKey.compareTo(current.element().key()) != 0) {    // Key 가 이미 존재하면 false

        // 현재 Key보다 더 작으므로 왼쪽 트리에 추가
        if(aKey.compareTo(current.element().key()) < 0) {

            // Left 없으면 그냥 추가
            if(current.left() == null) {
                current.setLeft(nodeForAdd);
                this.setSize(this.size() + 1);
                return true;
            }
        }
    }
}

```

```

    }

    // Left 있으면, 그 곳을 기준으로 하여 다시 따지기
    else
        current = current.left();
    }

    // 현재 Key 보다 더 크므로 오른쪽 트리에 추가
    else {

        // Right 없으면 그냥 추가
        if(current.right() == null) {
            current.setRight(nodeForAdd);
            this.setSize(this.size() + 1);
            return true;
        }

        // Right 있으면, 그 곳을 기준으로 하여 다시 따지기
        else
            current = current.right();
    }
}

// Key 가 이미 존재하여 while 벗어남
return false;
}

// Key 에 대응하는 Object 제거
@Override
public Obj removeObjectForKey(Key aKey) {

    // 비었음
    if(this.root() == null)
        return null;

    // 현재 root 에서 찾을
    if( aKey.compareTo(this.root().element().key()) == 0 ) {
        Obj objectForRemove = this.root().element().object();

        // left, right 모두 없음
        if( this.root().left() == null && this.root().right() == null )
            this.setRoot(null);

        // left 없고 right 있음 : right 로 대체
        else if( this.root().left() == null )
            this.setRoot(this.root().right());

        // left 있고 right 없음 : left 로 대체
        else if( this.root().right() == null )
            this.setRoot(this.root().left());

        // left, right 둘 다 있음 : Left tree 의 RightMost 삭제하여 대체
        else
            this.root().setElement( this.removeRightMostElementOfLeftSubTree(this.root()) );

        this.setSize(this.size() - 1);
        return objectForRemove;
    }
}

```

// root 에서 못찾음 -> 하위로 탐색

BinaryNode<DictionaryElement<Key, Obj>> current = **this.root()**;

BinaryNode<DictionaryElement<Key, Obj>> child = **null**;

do {

// 찾는 Key 가 현재 Key 보다 더 작음

if(aKey.compareTo(current.element().key()) < 0) {
 child = current.left();

if(child == **null**) {
 return null;
 }

// 왼쪽 서브트리의 root 에 찾는 Key 가 존재

if(aKey.compareTo(child.element().key()) == 0) {
 Obj objectForRemove = child.element().object();

 // left, right 모두 없음

if(child.left() == **null** && child.right() == **null**)
 current.setLeft(**null**);

 // left 없고 right 있음

else if(child.left() == **null**)
 current.setLeft(child.right());

 // left 있고 right 없음

else if(child.right() == **null**)
 current.setRight(child.left());

 // left, right 둘 다 있음 : child 의 Left tree 의 RightMost 삭제하여 대체

else
 child.setElement(**this.removeRightMostElementOfLeftSubTree**(child));

this.setSize(**this.size()** - 1);

return objectForRemove;

 }

}

// 찾는 Key 가 현재 Key 보다 더 큼

else {

 child = current.right();

if(child == **null**) {
 return null;
 }

// 오른쪽 서브트리의 root 에 찾는 Key 가 존재

if(aKey.compareTo(child.element().key()) == 0) {
 Obj objectForRemove = child.element().object();

 // left, right 모두 없음

if(child.left() == **null** && child.right() == **null**)
 current.setRight(**null**);

 // left 없고 right 있음

else if(child.left() == **null**)
 current.setRight(child.right());

 // left 있고 right 없음

else if(child.right() == **null**)


```

        current.setRight(child.left());

        // left, right 둘 다 있음 : child 의 Left tree 의 RightMost 삭제하여 대체
        else
            child.setElement(this.removeRightMostElementOfLeftSubTree(child));

        this.setSize(this.size() - 1);

        return objectForRemove;
    }
}

// 못찾음 -> 계속 하위로 이동
current = child;

} while(true);
}

// 사전 초기화
@Override
public void clear() {
    this.setSize(0);
    this.setRoot(null);
}

// 중위 탐색
public void scanInSortedorder() {
    this.inorderRecursively(this.root(), 1);
}

// 중위 탐색 : 거꾸로
public void scanReverseOfSortedOrder() {
    this.reverseOfInorderRecursively(this.root(), 1);
}

// 비공개 함수

// Key 에 대응하는 DictionaryElement 반환
private DictionaryElement<Key, Obj> elementForKey(Key aKey) {
    BinaryNode <DictionaryElement <Key, Obj>> current = this.root();
    while(current != null) {

        // 현재 노드에서 Key 찾을
        if(current.element().key().compareTo(aKey) == 0)
            return current.element();

        // 현재 노드의 Key 가 찾는 Key 보다 더 큼 -> 더 작은 곳 찾기
        else if(current.element().key().compareTo(aKey) > 0)
            current = current.left();

        // 현재 노드의 Key 가 찾는 Key 보다 더 작음 -> 더 큰 곳 찾기
        else
            current = current.right();
    }
}

```

```

    }

    // 못 찾음
    return null;
}

// 왼쪽 서브트리의 가장 오른쪽 Element 반환
// 왼쪽 서브트리가 존재할 때만 call됨
private DictionaryElement<Key, Obj> removeRightMostElementOfLeftSubTree
(BinaryNode<DictionaryElement<Key, Obj>> root)
{
    BinaryNode<DictionaryElement<Key, Obj>> leftOfRoot = root.left();

    // 왼쪽 서브트리의 right 에 아무 것도 없음 : 왼쪽 서브트리의 루트가 rightMost
    if(leftOfRoot.right() == null) {
        root.setLeft(leftOfRoot.left());          // 삭제
        return leftOfRoot.element();
    }

    // 왼쪽 서브트리의 right 가 존재 -> 탐색
    else {
        BinaryNode<DictionaryElement<Key, Obj>> parentOfRightMost = leftOfRoot;
        BinaryNode<DictionaryElement<Key, Obj>> rightMost = parentOfRightMost.right();

        // RightMost 까지 이동
        while(rightMost.right() != null) {
            parentOfRightMost = rightMost;
            rightMost = rightMost.right();
        }

        // 다음 rightMost : rightMost의 왼쪽 자식
        parentOfRightMost.setRight(rightMost.left());

        return rightMost.element();
    }
}

// 중위 탐색 : Call Back 함수 이용
private void inorderRecursively( BinaryNode<DictionaryElement<Key, Obj>> aRootOfSubtree, int aLevel ) {
    if(aRootOfSubtree != null) {
        this.inorderRecursively(aRootOfSubtree.left(), aLevel + 1);
        this.visitEvent().visitByCallBack(aRootOfSubtree.element(), aLevel);
        this.inorderRecursively(aRootOfSubtree.right(), aLevel + 1);
    }
}

// 거꾸로 중위 탐색 : Call Back 함수 이용
private void reverseOfInorderRecursively( BinaryNode<DictionaryElement<Key, Obj>> aRootOfSubtree, int aLevel ) {
    if(aRootOfSubtree != null) {
        this.reverseOfInorderRecursively(aRootOfSubtree.right(), aLevel + 1);
        this.visitEvent().visitInReverseOrder(aRootOfSubtree.element(), aLevel);
        this.reverseOfInorderRecursively(aRootOfSubtree.left(), aLevel + 1);
    }
}
}

```

```

public class DictionaryBySortedArray <Key extends Comparable<Key>, Obj>
    extends Dictionary <Key, Obj>
{

    // 상수
    private static final int DEFAULT_CAPACITY = 100000;

    // 인스턴스 변수
    private int _capacity;
    private DictionaryElement<Key, Obj>[] _elements;

    // Getter / Setter
    public int capacity() { return this._capacity; }
    private void setCapacity(int newCapacity) { this._capacity = newCapacity; }

    // 생성자
    public DictionaryBySortedArray() {
        this(DictionaryBySortedArray.DEFAULT_CAPACITY);
    }

    @SuppressWarnings("unchecked")
    public DictionaryBySortedArray(int givenCapacity) {
        this.setCapacity(givenCapacity);
        this._elements = new DictionaryElement[this.capacity()];
    }

    // 다 찼는지 확인
    @Override
    public boolean isFull() {
        return ( this.size() == this.capacity() );
    }

    // Key 의 존재 여부 확인
    @Override
    public boolean keyDoesExist(Key aKey) {
        int keyPosition = this.positionFor(aKey);
        if(keyPosition < 0)
            return false;
        else
            return true;
    }

    // Key 에 대응하는 Object 탐색
    @Override
    public Obj objectForKey(Key aKey) {
        int positionWithKey = this.positionFor(aKey);

        // Key에 대응하는 원소가 존재하지 않음
        if(positionWithKey < 0)
            return null;

        // Key에 대응하는 원소가 존재함
        return this._elements[positionWithKey].object();
    }
}

```

```

// Key 와 Object 쌍 추가
@Override
public boolean addKeyAndObject(Key aKey, Obj anObject) {
    int positionForAdd = this.positionFor(aKey);

    // 이미 존재 : positionFor() 가 양수 위치를 반환
    if(positionForAdd >= 0)
        return false;

    // 존재 x : positionFor() 가 음수 위치를 반환
    // positionFor()에서 음수화하였던 위치를 원래대로 되돌려주면 정상적인 추가 위치 나옴
    positionForAdd = -(positionForAdd + 1);
    this.makeRoomAt(positionForAdd);
    this._elements[positionForAdd] = new DictionaryElement<Key, Obj>(aKey, anObject);
    this.setSize(this.size() + 1); // Getter 를 통한 크기 증가 : 부모 클래스의 private 필드이기 때문

    return true;
}

```

```

// Key 에 대응하는 Object 제거
@Override
public Obj removeObjectForKey(Key aKey) {
    int positionForRemove = this.positionFor(aKey);

    // Key에 대응하는 원소가 존재하지 않음
    if(positionForRemove < 0) {
        return null;
    }

    // Key에 대응하는 원소가 존재함
    Obj removedObject = this._elements[positionForRemove].object();
    this.removeGapAt(positionForRemove);
    this.setSize(this.size() - 1);
    return removedObject;
}

```

```

// 사전 초기화
@Override
public void clear() {
    this.setSize(0);
}

```

// 비공개 함수

```

// 이진 탐색으로 Key가 존재하는 위치 탐색, 없으면 음수화하여 반환
private int positionFor(Key aKey) {
    int left = 0;
    int right = this.size() - 1;

    while(left <= right) {
        int mid = (left + right) / 2;

        switch( aKey.compareTo(this._elements[mid].key()) ) {

```

```

        case -1:
            right = mid - 1;
            break;
        case 0:
            return mid;
        case 1:
            left = mid + 1;
            break;
    }
}

// 못 찾음
return -(left + 1); // left가 0 일 수도 있기 때문에 1을 더한 후 음수화
// 역계산하면 Key에 알맞는 위치 계산 가능 : 추가할

```

때 사용

```

}

```

```

// 배열의 특정 위치에 공간 생성
private void makeRoomAt(int aPosition) {
    for(int i = this.size(); i > aPosition; i--)
        this._elements[i] = this._elements[i - 1];
}

```

```

// 배열의 특정 위치의 공간 제거
private void removeGapAt(int aPosition) {
    for(int i = aPosition; i < this.size() - 1; i++)
        this._elements[i] = this._elements[i + 1];
}

```

```

}

```

DictionaryBySortedLinkedList

```

public class DictionaryBySortedLinkedList <Key extends Comparable<Key>, Obj>
    extends Dictionary <Key, Obj>
{

    // 인스턴스 변수
    private ListNode<DictionaryElement <Key, Obj>> _head;

    // Getter / Setter
    private ListNode<DictionaryElement<Key, Obj>> head() { return this._head; }
    private void setHead(ListNode<DictionaryElement<Key, Obj>> newHead) { this._head = newHead; }

    // 생성자
    public DictionaryBySortedLinkedList() {
        this.clear();
    }

    // 다 찾는지 확인
    @Override
    public boolean isFull() {
        return false;
    }
}

```

```

// Key 의 존재 여부 확인
@Override
public boolean keyExists(Key aKey) {
    ListNode<DictionaryElement<Key, Obj>> current = this.head();

    while(current != null) {

        // 오름차순 리스트이므로, 순차적으로 탐색하면서 대상 Key를 만나기 전에 더 큰 것을 만난다면
        // 대상보다 더 작음
        switch( current.element().key().compareTo(aKey) ) {

            case -1:
                current = current.next();
                break;

            // 대상 찾음
            case 0:
                return true;

            // 대상보다 더 큼 : 리스트에 없음
            case 1:
                return false;

        }

        // 끝까지 가도 못찾음
        return false;
    }
}

```

없는거임

```

// Key 에 대응하는 Object 탐색
@Override
public Obj objectForKey(Key aKey) {
    ListNode<DictionaryElement<Key, Obj>> current = this.head();

    // 오름차순이므로, 순차적으로 탐색하면서 대상 Key를 만나기 전에 더 큰 것을 만난다면 없는거임
    while( (current != null) && (current.element().key().compareTo(aKey)) < 0 )
        current = current.next();

    // 찾음
    if( (current != null) && (current.element().key().compareTo(aKey) == 0) )
        return current.element().object();

    // 못찾음
    else
        return null;
}

```

```

// Key 와 Object 쌍 추가
@Override
public boolean addKeyAndObject(Key aKey, Obj anObject) {
    ListNode<DictionaryElement<Key, Obj>> previous = null;
    ListNode<DictionaryElement<Key, Obj>> current = this.head();

    // 현재 Key가 추가할 Key보다 더 작음 -> 이동
    while( (current != null) && (current.element().key().compareTo(aKey) < 0) ) {
        previous = current;
        current = current.next();
    }
}

```

```

    }

    // 현재 key가 추가할 key와 같음 -> 이미 있으므로 추가 못함
    if( (current != null) && (current.element().key().compareTo(aKey) == 0) )
        return false;

    // 추가할 노드 생성
    DictionaryElement<Key, Obj> addedElement = new DictionaryElement<Key,Obj> (aKey, anObject);
    ListNode< DictionaryElement<Key, Obj> > addedNode = new ListNode< DictionaryElement<Key, Obj> >();
    addedNode.setElement(addedElement);

    // 추가 (연결)
    addedNode.setNext(current);
    if(previous == null) // 맨 앞에 추가
        this.setHead(addedNode);
    else // 중간에 추가
        previous.setNext(addedNode);

    // size 증가
    this.setSize(this.size() + 1); // Getter 를 통한 크기 증가 : 부모 클래스의 private 필드이기 때문

    return true;
}

// Key 에 대응하는 Object 제거
@Override
public Obj removeObjectForKey(Key aKey) {
    ListNode< DictionaryElement<Key, Obj> > previous = null;
    ListNode< DictionaryElement<Key, Obj> > current = this.head();

    // 현재 key가 제거할 key보다 더 작음 -> 이동
    while( (current != null) && (current.element().key().compareTo(aKey) < 0) ) {
        previous = current;
        current = current.next();
    }

    // 현재 key가 제거할 key와 같음 -> 제거
    if( (current != null) && (current.element().key().compareTo(aKey) == 0) ) {

        // 맨 앞 제거
        if(current == this.head())
            this.setHead(current.next());

        // 중간 제거
        else
            previous.setNext(current.next());

        // size 증가
        this.setSize(this.size() - 1);

        // 제거된 Object 반환
        return current.element().object();
    }

    // 없음
    else

```

```

        return null;
    }

    // 사전 초기화
    @Override
    public void clear() {
        this.setSize(0);
        this.setHead(null);
    }
}

```

DictionaryElement.java

```

public class DictionaryElement<Key, Obj> {

    private Key _key;
    private Obj _object;

    // 생성자
    public DictionaryElement() { }
    public DictionaryElement(Key givenKey, Obj givenObject) {
        this._key = givenKey;
        this._object = givenObject;
    }

    // Getter / Setter
    public Key key() { return this._key; }
    public void setKey(Key newKey) { this._key = newKey; }

    public Obj object() { return this._object; }
    public void setObject(Obj newObject) { this._object = newObject; }
}

```

LinkedListNode.java

```

public class LinkedListNode<E> {

    private E _element;
    private LinkedListNode<E> _next;

    // Getter / Setter
    public E element() { return this._element; }
    public void setElement(E newElement) { this._element = newElement; }

    public LinkedListNode<E> next() { return this._next; }
    public void setNext(LinkedListNode<E> newNext) { this._next = newNext; }

    // 생성자
    public LinkedListNode() { }
    public LinkedListNode(E givenElement, LinkedListNode<E> givenNext) {
        this._element = givenElement;
        this._next = givenNext;
    }
}

```

ListOrder.java


```

public enum ListOrder {

    Ascending,
    Descending,
    Random;

    public static final String[] STRINGS_IN_KOREAN = new String[] {"오름차순", "내림차순", "무작위"};

    public String toStringInKorean() {
        return ListOrder.STRINGS_IN_KOREAN[this.ordinal()];
    }

}

```

ParameterSet.java

```

public class ParameterSet {

    // Private instance variables
    private int _firstDataSize;
    private int _numberOfExperiments;
    private int _sizeIncrement;

    // Getters & Setters
    public int firstDataSize() { return this._firstDataSize; }
    public void setFirstDataSize(int newFirstDataSize) { this._firstDataSize = newFirstDataSize; }

    public int numberOfExperiments() { return this._numberOfExperiments; }
    public void setNumberOfExperiments(int newNumberOfExperiments) {
        this._numberOfExperiments = newNumberOfExperiments;
    }

    public int sizeIncrement() { return this._sizeIncrement; }
    public void setSizeIncrement(int newSizeIncrement) { this._sizeIncrement = newSizeIncrement; }

    // Constructor
    public ParameterSet( int givenNumberOfExperiments,
                        int givenFirstDataSize,
                        int givenSizeIncrement )
    {
        this._numberOfExperiments = givenNumberOfExperiments;
        this._firstDataSize = givenFirstDataSize;
        this._sizeIncrement = givenSizeIncrement;
    }

    // return Maximum Data Size
    public int maxDataSize() {
        return this.firstDataSize() + ( this.sizeIncrement() * ( this.numberOfExperiments() - 1 ) );
    }

}

```

PerformanceMeasurement.java

// Manages Experiment with Required Data

```

public class PerformanceMeasurement {

    // Constants for the Parameters :
    private static final int DEFAULT_NUMBER_OF_EXPERIMENTS = 5;
    private static final int DEFAULT_SIZE_INCREMENT = 10000;
    private static final int DEFAULT_FIRST_DATA_SIZE = 10000;

```

```

// Instance variables :
private ParameterSet          _parameterSet;          // Parameter Set for test settings

private int[]                 _ascendingList;          // Ascending Data List to Sort
private int[]                 _descendingList;        // Descending Data List to Sort
private int[]                 _randomList;            // Random Data List to Sort


// Getters / Setters
public ParameterSet parameterSet() { return this._parameterSet; }
public void setParameterSet(ParameterSet newParameterSet) { this._parameterSet = newParameterSet; }

public int[] ascendingList() { return this._ascendingList; }
public int[] descendingList() { return this._descendingList; }
public int[] randomList() { return this._randomList; }


// Constructors
public PerformanceMeasurement() {
    this.setParameterSet( new ParameterSet( PerformanceMeasurement.DEFAULT_NUMBER_OF_EXPERIMENTS,
PerformanceMeasurement.DEFAULT_FIRST_DATA_SIZE,
PerformanceMeasurement.DEFAULT_SIZE_INCREMENT ) );
    this.generateData();
}

public PerformanceMeasurement( int givenNumberOfExperiments,
                                int givenFirstDataSize,
                                int givenSizeIncrement )
{
    this.setParameterSet( new ParameterSet( givenNumberOfExperiments,
givenFirstDataSize,
givenSizeIncrement ) );
    this.generateData();
}


// 데이터 생성
public void generateData() {
    this._ascendingList = DataGenerator.ascendingList(this.parameterSet().maxDataSize());
    this._descendingList = DataGenerator.descendingList(this.parameterSet().maxDataSize());
    this._randomList = DataGenerator.randomList(this.parameterSet().maxDataSize());
}


// 주어진 List Order에 맞는 데이터 리스트 반환
public int[] experimentList(ListOrder anOrder) {
    if(anOrder == ListOrder.Ascending)
        return this._ascendingList;

    else if(anOrder == ListOrder.Descending)
        return this._descendingList;

    else if(anOrder == ListOrder.Random)
        return this._randomList;

    else
        return null;
}

```

```

// 데이터 크기 변화시키며 측정 실험 실행
// Dictionary 종류, 데이터 종류 설정
public UnitExperimentResult[] experimentByDictionaryAndListOrderType
(Dictionary<Integer, String> aDictionary,
ListOrder anOrder)

{
    UnitExperimentResult[] experimentResults =
        new UnitExperimentResult[this.parameterSet().numberOfExperiments()];

    // 측정 결과의 안정화를 위한 테스트, 측정과는 무관
    this.unitExperiment(aDictionary, anOrder, this.parameterSet().maxDataSize());
    System.gc(); // Garbage Collection 강제 실시 : 측정 실행 동안에 쓸데없이 발생하지 않도록

    // 측정
    int dataSize = this.parameterSet().firstDataSize();
    for(int iteration = 0; iteration < this.parameterSet().numberOfExperiments(); iteration++) {
        aDictionary.clear();
        experimentResults[iteration] = this.unitExperiment(aDictionary, anOrder, dataSize);
        dataSize += this.parameterSet().sizeIncrement();
    }

    return experimentResults;
}

```

// 비공개 함수

// 측정 실험 후 결과 반환

// Dictionary 종류, 데이터 종류, 데이터 크기 설정

```

private UnitExperimentResult unitExperiment(Dictionary<Integer, String> aDictionary,
anOrder,
ListOrder
int dataSize)

{
    long startTime;
    long stopTime;
    int[] experimentList = this.experimentList(anOrder); // 데이터 종류 설정

    // 주어진 데이터 크기만큼 추가하는 데 걸리는 시간 측정
    long timeForAdd = 0;
    for(int i = 0; i < dataSize; i++) {
        startTime = System.nanoTime();
        aDictionary.addKeyAndObject(experimentList[i], null); // 사전 종류에 맞는 추가
        stopTime = System.nanoTime();
        timeForAdd += (stopTime - startTime);
    }
}

```

// 주어진 데이터 크기와 같은 횟수 탐색하는 데 걸리는 시간 측정

```

        long timeForSearch = 0;
        for(int i = 0; i < dataSize; i++) {
            startTime = System.nanoTime();
            aDictionary.objectForKey(experimentList[i]);
            stopTime = System.nanoTime();
            timeForSearch += (stopTime - startTime);
        }

        // 주어진 데이터 크기만큼 제거하는 데 걸리는 시간 측정
        long timeForRemove = 0;
        for(int i = 0; i < dataSize; i++) {
            startTime = System.nanoTime();
            aDictionary.removeObjectForKey(experimentList[i]);
            stopTime = System.nanoTime();
            timeForRemove += (stopTime - startTime);
        }

        return ( new UnitExperimentResult (dataSize, timeForAdd, timeForSearch, timeForRemove) );
    }
}

```

UnitExperimentResult.java

```

public class UnitExperimentResult {

    private int _experimentSize;
    private long _timeForAdd;
    private long _timeForSearch;
    private long _timeForRemove;

    // 생성자
    public UnitExperimentResult() {}
    public UnitExperimentResult( int givenExperimentSize,
                                long givenTimeForAdd,
                                long givenTimeForSearch,
                                long givenTimeForRemove )
    {
        this._experimentSize = givenExperimentSize;
        this._timeForAdd = givenTimeForAdd;
        this._timeForSearch = givenTimeForSearch;
        this._timeForRemove = givenTimeForRemove;
    }

    // Getter / Setter

    public int experimentSize() { return this._experimentSize; }
    public void setExperimentSize(int newExperimentSize) { this._experimentSize = newExperimentSize; }

    public long timeForAdd() { return this._timeForAdd; }
    public void setTimeForAdd(long newTimeForAdd) { this._timeForAdd = newTimeForAdd; }

    public long timeForSearch() { return this._timeForSearch; }
    public void setTimeForSearch(long newTimeForSearch) { this._timeForSearch = newTimeForSearch; }

    public long timeForRemove() { return this._timeForRemove; }
    public void setTimeForRemove(long newTimeForRemove) { this._timeForRemove = newTimeForRemove; }
}

```

```
}
```

VisitEventSourceForTreeTraversal.java

```
public interface VisitEventSourceForTreeTraversal<Key, Obj> {  
    public void visitByCallBack(DictionaryElement<Key, Obj> anElement, int aLevel);  
    public void visitInReverseOrder(DictionaryElement<Key, Obj> anElement, int aLevel);  
}
```

DS1_14_201702081_최재범.java

```
public class DS1_14_201702081_최재범 {  
  
    public static void main(String[] args) {  
  
        ApplicationController_TreeTraversal app_T = new ApplicationController_TreeTraversal();  
        app_T.run();  
  
        ApplicationController_PerformanceComparement app_P = new ApplicationController_PerformanceComparement();  
        app_P.run();  
  
    }  
}
```